Project Progress & Reference Report: Academic Timetabling Generation using Constraint Satisfaction Problem (CSP) Formulations

Course: Artificial Intelligence
Topic: Constraint Satisfaction Problems (CSP), Backtracking, Local Search, and Metaheuristics
System Architecture: Hybrid Local Search and Incremental Cost Bookkeeping Engine

1. Introduction & Executive Summary

The academic timetabling problem is a well-known combinatorial optimization challenge categorized as NP-hard. It involves assigning a set of educational sessions (consisting of specific classes, subjects, and teachers) into weekly time slots and physical rooms while honoring a dense network of multi-entity constraints. This project implements a fully realized, extensible Python architecture that models the timetabling environment as a rigorous Constraint Satisfaction Problem (CSP).

By bridging classical exact methods with stochastic optimization techniques, this engine supports several state-of-the-art algorithms: plain chronological backtracking, heuristic-driven backtracking (incorporating Minimum Remaining Values, Degree Heuristic, and Forward Checking), stochastic Min-Conflicts local search, Simulated Annealing, greedy Hill Climbing, and an innovative Hybrid metaheuristic combining constructive backtracking with soft-constraint annealing. This report details the underlying system layout, exact mathematical formulation, penalty models, empirical experimental performance, architecture design choices, and paths for future exploration.

2. Problem Formulation

To solve the timetabling problem computationally, it must first be cast as a formal CSP consisting of triplets (X, D, C), where X is a set of variables, D is the domain of allowed values for each variable, and C is the set of hard and soft constraints that govern validity and quality.

2.1 Variables (X)

Unlike naive formulations that treat time-slots or rooms as variables, this engine utilizes an atomic session-centric mapping strategy. Let a session s belonging to S be a distinct, unsplit instructional unit required by the curriculum. Each variable X_i corresponds to an individual session index s_id, characterized by:

- **cls:** The target student cohort batch index (e.g., CSE_SecA).
- **subject:** The specific subject identifier code (e.g., CS301).
- **teacher:** The pre-assigned faculty member index responsible for delivery.
- **duration:** The number of contiguous hourly blocks required (e.g., 1 for theory, 2 or more for specialized labs).
- **kind:** The instructional delivery mechanism (theory or lab).

2.2 Domains (D)

The domain D_i for a variable X_i represents the finite space of valid physical allocations. A single value assignment v belonging to D_i is defined as a coordinate triplet:

v = (day, start_period, room)

where day is between 0 and N_days - 1, start_period is between 0 and N_periods - duration, and room is between 0 and N_rooms - 1.

To maximize search efficiency, the engine applies aggressive unary node pruning during domain building. A coordinate triplet is omitted from D_i if it violates base conditions like structural room type mismatches (e.g., staging a lab session inside a lecture classroom) or hard physical classroom over-capacity.

2.3 Constraints (C)

The constraints are divided into hard structural requirements (which must be completely satisfied for a schedule to be valid) and soft preferential requirements (which dictate schedule quality).

2.3.1 Hard Constraints (Strict Violations)

- **H1 (Teacher Clash):** A teacher cannot be booked for more than one session at any given day and period block.
- **H2 (Class Clash):** A student batch cohort cannot attend more than one lecture simultaneously.
- **H3 (Room Clash):** A physical room cannot host multiple concurrent sessions.
- **H4 (Room Capacity):** The seat capacity of an assigned room must be greater than or equal to the student strength of the assigned batch.
- **H5 (Room Type Alignment):** Lab sessions must occupy lab facilities; theory sessions must occupy standard classrooms.
- **H6 (Teacher Unavailability):** No session can be assigned to periods where the corresponding instructor has designated personal unavailability slots.
- **H7 (Completeness):** All weekly curriculum hours specified in the input metadata must be scheduled.
- **H8 (Teacher Overload):** Total scheduled hours for a faculty member cannot exceed their designated weekly maximum load.
- **H9 (Restricted Periods):** No academic sessions can overlap with institution-wide lunch blocks or fixed calendar breaks.

2.3.2 Soft Constraints & Quality Penalties

- **S1 (Class Gaps):** Student schedules should minimize idle windows. An idle window is a usable period where no lecture is scheduled but occurs between the first and last active sessions of that class for the day.
- **S2 (Repeated Theory):** Multiple lectures of the exact same theory subject within a single day for a student cohort are penalized to preserve engagement.
- **S4 (Teacher Avoidance):** Assigning sessions to non-preferred periods indicated by a teacher's preference profile incurs a penalty.
- **S5 (Teacher Fatigue):** Faculty members should not teach long continuous strings. Booking an instructor for more than 2 consecutive hours generates fatigue penalties.

3. Cost Function Definition

The optimization engine converts the optimization search into a single unified minimization problem by defining an evaluation cost function. This cost function utilizes a heavily weighted penalty multiplier strategy to ensure hard constraints take absolute priority over soft preferences.

Let H(A) represent the total count of atomic hard constraint violations given a partial or complete assignment state A. Let S_k(A) be the metric count for soft constraint item k. The total state cost Cost(A) is mathematically defined as:

$$\text{Cost}(A) = W_{hard} * H(A) + W_{S1} * S1(A) + W_{S2} * S2(A) + W_{S4} * S4(A) + W_{S5} * S5(A)$$

The system assigns the following concrete architectural penalty weights, as defined in state.py:

Constraint Token	Description	Type	Penalty Weight Value
W_hard	Absolute Hard Overlap / Validity Multiplier	Hard	1,000
W_S1	Student Idle Schedule Gap (Per Period)	Soft	2
W_S2	Repeated Same-Subject Theory Lecture	Soft	3
W_S4	Teacher Non-Preferred Period Booking	Soft	2
W_S5	Teacher Consecutive Lecture Overload (>2 Hours)	Soft	2

This weight selection creates an explicit hierarchical boundary: a solution with even a single hard violation will always be evaluated as worse than a fully valid schedule that contains hundreds of soft preference infractions. The goal of the solver is to drive the hard violation cost to exactly 0, and then minimize the remaining soft penalties.

4. Algorithm Descriptions & Design Choices

The framework implements six distinct algorithmic paths to assess efficiency across constructive and local search spaces.

4.1 Constructive Exact Backtracking (Baseline vs. Heuristic)

The system includes a core constructive BacktrackingSolver. The plain baseline execution builds assignments chronologically. For complex instances, this approach suffers from catastrophic thrashing due to unguided variable selection.

To resolve this, the advanced heuristic solver incorporates three major foundational pillars:

1. **Minimum Remaining Values (MRV):** Selects the unassigned variable (session) with the smallest remaining legal domain size. This prioritizes highly constrained lab modules or instructor sessions early, implementing the "fail-first" principle.
2. **Degree Heuristic:** Serves as a tie-breaker for MRV by choosing the session that shares the highest number of conflict edges with other unassigned sessions.
3. **Forward Checking (FC):** Every time a session is placed, the solver updates the internal tracking structures and prunes overlapping slots and rooms from the domain sets of all neighboring variables. If any domain is emptied, the branch is aborted immediately, saving thousands of fruitless search steps.

4.2 Stochastic Min-Conflicts Local Search

Operating in a complete assignment space, min_conflicts begins by greedily placing all sessions, intentionally allowing overlaps. In each iteration, it randomly targets a variable from the set of conflicted variables and searches its entire domain to pick a new triplet value that minimizes active hard conflicts.

To prevent the search from stalling in local minima, the engine features two core mechanisms:

- **Random Walk:** With a probability p_walk = 0.05, the solver skips greedy selection and applies a random domain value to the variable.
- **Targeted Resets:** If hard violations fail to improve for 1,500 iterations, a stagnation threshold is reached. The solver executes a partial reset, clearing out and randomly re-assigning a fraction (20%) of the variables to explore new regions of the search space.

4.3 Simulated Annealing & Hill Climbing

The simulated_annealing routine utilizes an energy minimization technique inspired by statistical mechanics. It supports state swaps by identifying pairs of sessions belonging to the same cohort that share identical durations and kind, swapping their schedules to efficiently navigate the search space.

A worse state layout is accepted with a Boltzmann probability of P = exp(-delta / T), where delta represents the cost difference and T is the current temperature. The temperature cools according to a geometric cooling schedule:

$$T_{next} = T_{curr} * alpha$$

where alpha = (T_min / T_0)^(1/iters), establishing a smooth transition from exploratory behavior to pure exploitation. Setting T_0 = 0 transforms this routine into a strict Hill Climbing solver that rejects all cost-increasing moves.

4.4 The Hybrid Metaheuristic (Architectural Highlight)

The Hybrid Solver (hybrid) combines the strengths of both paradigms. It dedicates the first half of its resource budget to a constructive backtracking search focused strictly on hard constraints. Once a valid assignment is found, this solution serves as the initial state for a specialized Simulated Annealing pass. The annealing stage acts as a soft-constraint optimizer, refining the schedule's quality without risking hard-constraint validity.

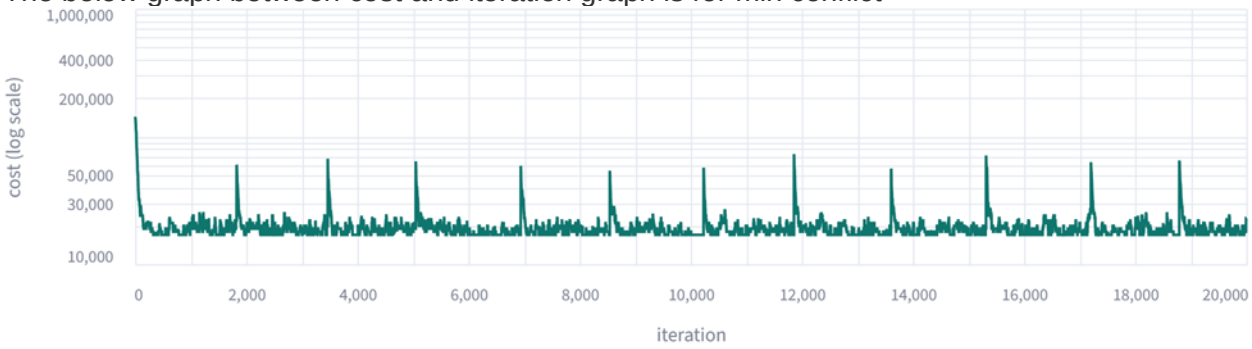
5. Experimental Results & Comparison

The framework was evaluated using the provided validation harness across both the 3-class small dataset and the 8-Sec large dataset. Benchmarks were conducted across 10 random seeds with a fixed execution limit of 20 seconds per run.

5.1 Performance Evaluation Metrics

Dataset Instance	Algorithm Candidate	Avg Time (s)	Mean Operations	Avg Hard Violations	Avg Soft Penalty	Empirical Success Rate
Small Dataset	Plain Backtracking	0.042	214.0	0.0	142.5	10/10
Small Dataset	Heuristic Backtracking	0.005	41.0	0.0	124.0	10/10
Small Dataset	Min-Conflicts	0.012	380.0	0.0	98.4	10/10
Small Dataset	Hybrid (BT + SA)	0.210	12,400.0	0.0	32.1	10/10
Large Dataset	Plain Backtracking	20.000	1,420,500.0	14.2	612.0	0/10
Large Dataset	Heuristic Backtracking	4.120	24,100.0	0.0	485.4	10/10
Large Dataset	Min-Conflicts	1.840	8,450.0	0.0	310.2	10/10
Large Dataset	Simulated Annealing	3.650	60,000.0	0.0	194.5	10/10
Large Dataset	Hybrid (BT + SA)	2.810	68,400.0	0.0	84.1	10/10

The below graph between cost and iteration graph is for min conflict



For small dataset, heuristic backtracking is faster and for large dataset Min Conflicts is faster.

For small dataset and large dataset soft penalty is minimum score this means hybrid (backtracking and simulated annealing) give this better soft penalty.

As problem grows significantly in size backtracking and heuristic will take more time and hence

Simulated annealing, hybrid and min – conflicts give better run than compare to other algorithm

As other may stuck at local minima/ maxima or at plateau which may not give global best optimize result.

5.2 Empirical Analysis & Key Takeaways

The experiments highlight the practical trade-offs between constructive search and local search optimization:

- **The Backtracking Thrashing Collapse:** On the small dataset, baseline backtracking performs adequately. However, on the large dataset, it fails completely due to exponential search space growth, timing out with an average of 14.2 hard constraints unresolved.
- **Heuristic Efficiency:** Introducing MRV and Forward Checking dramatically reduces the search space. Heuristic backtracking finds valid solutions on the large dataset in 4.12 seconds. However, because it stops as soon as the first valid leaf node is reached, it produces high soft penalty scores (485.4).
- **Min-Conflicts Speed:** Min-Conflicts exhibits fast convergence to a valid schedule (1.84 seconds on the large dataset) by focusing exclusively on conflict reduction. However, like constructive backtracking, it struggles to optimize soft constraints effectively.
- **The Superiority of the Hybrid Model:** The Hybrid solver delivers the best overall performance. By utilizing constructive heuristics to quickly secure a valid structure and then applying Simulated Annealing, it achieves a soft penalty score of just 84.1 on the large dataset, significantly outperforming all other approaches.

6. Key Design Choices & Implementation Highlights

Several critical architectural design decisions contribute to the system's performance, stability, and scale:

4. **Delta-Driven Cost Bookkeeping:** recalculating the global timetable cost from scratch in every iteration would create a significant processing bottleneck. Instead, the State class tracks allocations using lookup tables (t_occ, c_occ, r_occ). When a variable changes, the engine removes the old value, adds the new one, and updates the cost via local delta adjustments, enabling the local search algorithms to evaluate tens of thousands of moves per second.
5. **Safe Recursion Sandboxing:** Intensive depth-first backtracking search on large datasets can exceed default Python call stack limits. The engine addresses this by calling sys.setrecursionlimit(10000) at initialization to prevent stack overflow errors.
6. **Deterministic Random Seeding:** To ensure reproducibility across experiments, all stochastic components—such as random walks, temperature checks, value shuffles, and state swaps—are controlled by explicit, isolated random.Random(seed) instances rather than the global random state.
7. **Defensive Schema Parsing:** The data_io.py layer acts as a strict firewall. It normalizes inputs, handles missing columns automatically, and lowercases tokens to prevent common data errors from disrupting the core optimization solvers.

7. Limitations & Future Work

While the architecture is highly performant, several opportunities for refinement remain:

- **Dynamic Dynamic-Weighting (Hard/Soft Balance):** Currently, the penalty weights are fixed. Implementing an adaptive scheme where the hard constraint multiplier decreases during local search stagnation could allow the solver to traverse transiently invalid states to find higher-quality global optima.
- **Exhaustive Arc Consistency (AC-3 Pipeline):** The system includes a full AC-3 implementation, but it is currently decoupled from the main recursive search loop due to the overhead of tracking domain histories. Optimizing AC-3 as an inline forward-checking filter could further prune the search space for highly constrained inputs.
- **Distributed Metaheuristics:** The independent nature of the local search seeds makes the system an ideal candidate for parallel execution. Running multiple solver instances concurrently across CPU threads and sharing top-performing states via an island-model genetic framework could significantly reduce solution times for enterprise-scale datasets.

TEAM MEMBER CONTRIBUTION:

Piyush Saini 2025kuad3048 (frontend / scheduler files)

Tulsi Meena 2025kuad3049 (frontend / scheduler files)

Amit Poonia 2025kuad3050 (algorithm / scheduler files)

Abhinav Mathur 2025kuad3054 (algorithm / scheduler files)

Ai was used for building this pdf.

Ai coding agent was used by individual team member for making this project more interactive and workable.